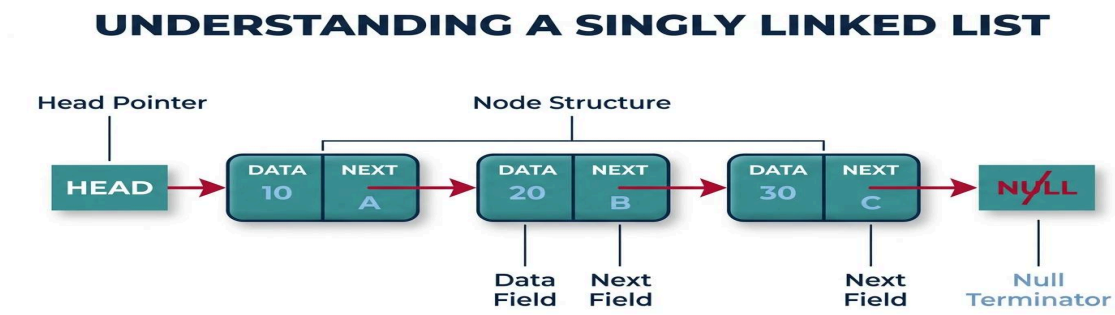
Introduction to Linked List



A Linked List is a linear data structure in which elements are stored in separate memory locations called nodes.

Each node contains:

1. Data – the value stored in the node.
2. Next – pointer to the next node.

Unlike arrays, linked-list elements are stored in contiguous memory locations.

C++ node structure

None

```
struct Node {
    int data;
    Node* next;

    Node(int val) {
        data = val;
        next = NULL;
    }
};
```

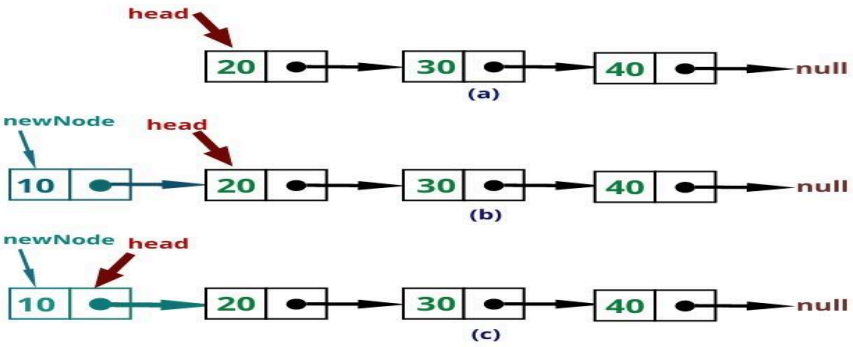
Important terminology

- Head: Pointer to the first node.
- Tail: Last node, whose `next` is `NULL`.

- NULL: Indicates that there is no next node.

Insertion in Linked List

A. Insertion at Head



Steps:

1. Create a new node.
2. Point its `next` to the current head.
3. Update head to the new node.

```C/C++
void insertAtHead(Node* &head, int val) {
    Node* newNode = new Node(val);

    newNode->next = head;
    head = newNode;
}
```

Time Complexity: O(1)

B. Insertion at Tail

Suppose:

`10 → 20 → 30 → NULL`

Insert `40` at the end.

Steps:

1. Create a new node.
2. If the list is empty, make it the head.
3. Otherwise, traverse until the last node.
4. Connect the last node to the new node.

```C/C++
void insertAtTail(Node* &head, int val) {
    Node* newNode = new Node(val);

    if (head == NULL) {
        head = newNode;
        return;
    }

    Node* temp = head;

    while (temp->next != NULL) {
        temp = temp->next;
    }

    temp->next = newNode;
}
```

Time Complexity: O(n)

C. Insertion in Middle / At a Position

Suppose:

`10 → 20 → 30 → NULL`

Insert `25` at position 3 (using 1-based indexing).

Steps:

1. Reach the node just before the required position.
2. Create a new node.
3. Connect the new node to the next node.
4. Connect the previous node to the new node.

C/C++

```
void insertAtPosition(Node* &head, int pos, int val) {
    if (pos < 1) return;

    if (pos == 1) {
        insertAtHead(head, val);
        return;
    }

    Node* temp = head;

    for (int i = 1; i < pos - 1 && temp != NULL; i++) {
        temp = temp->next;
    }

    if (temp == NULL) return;

    Node* newNode = new Node(val);

    newNode->next = temp->next;
    temp->next = newNode;
}
```

Time Complexity: O(n)

Deletion in Linked List

A. Delete from Head

Example:

```
10 → 20 → 30 → NULL
```

Delete the first node.

C/C++

```
void deleteHead(Node* &head) {
    if (head == NULL) return;

    Node* temp = head;
    head = head->next;

    delete temp;
}
```

Time Complexity: O(1)

B. Delete from Tail

Example:

`10 → 20 → 30 → NULL`

Delete `30`.

C/C++

```
void deleteTail(Node* &head) {
    if (head == NULL) return;

    if (head->next == NULL) {
        delete head;
        head = NULL;
        return;
    }

    Node* temp = head;

    while (temp->next->next != NULL) {
        temp = temp->next;
    }

    delete temp->next;
    temp->next = NULL;
```

```
}
```

Time Complexity: O(n)

Why do we use `temp->next->next != NULL`?

Because we need to stop at the second-last node so that we can delete the last node and update the second-last node's `next` to `NULL`.

C. Delete from Middle / Position

C/C++

```
void deleteAtPosition(Node* &head, int pos) {
    if (head == NULL || pos < 1) return;

    if (pos == 1) {
        deleteHead(head);
        return;
    }

    Node* temp = head;

    for (int i = 1; i < pos - 1 && temp != NULL; i++) {
        temp = temp->next;
    }

    if (temp == NULL || temp->next == NULL) return;

    Node* nodeToDelete = temp->next;
    temp->next = nodeToDelete->next;

    delete nodeToDelete;
}
```

Time Complexity: O(n)

Operation	Time Complexity	Description
Insertion at Head	`O(1)`	Add new node at beginning by updating head pointer
Insertion at Tail	`O(n)`	Traverse to end and add new node (O(1) with tail pointer)
Deletion at Head	`O(1)`	Remove first node by updating head pointer
Deletion by Value	`O(n)`	Traverse list to find and remove specific node
Search	`O(n)`	Traverse list to find element
Access by Index	`O(n)`	Traverse list until reaching desired position